

Client-Side Performance

Response Caching, ETags, and Global Profiles

1. The Paradigm Shift: Output vs. Response Caching

To master API performance, you must understand exactly *where* the data is being stored.

Feature	Output Caching (Server-Side)	Response Caching (Client-Side)
Where does it live?	On your API Server's RAM (or Redis).	In the user's Browser or a CDN (Cloudflare).
Who controls it?	The Server.	The Client (via HTTP Headers).
Best Used For...	Heavy DB queries that are identical for all users.	Static assets, or data specific to one user to save network bandwidth.

2. ETags (Entity Tags) & Bandwidth Optimization

An **ETag** is a unique fingerprint (usually a hash) of the requested data.

The 304 Not Modified Flow:

- First Request:** Server sends 5MB of JSON and an ETag: "hash-123".
- Second Request:** Client asks for data again, sending If-None-Match: "hash-123".
- Server Check:** Server sees the hash matches the current database state.
- Result:** Server sends a 304 Not Modified with an **EMPTY body**. Bandwidth saved!

3. Advanced Cache-Control Headers

When dealing with Response Caching, you are manipulating standard HTTP Cache-Control headers.

Security Warning: Public vs. Private

Never use `Cache-Control: public` on endpoints that return sensitive user data (like a bank balance). Intermediate network routers and CDNs will save the data, exposing User A's data to User B. ALWAYS use `Cache-Control: private` for authenticated user data.

Directive	What it tells the Browser / CDN
<code>max-age=60</code>	"Keep this in your hard drive for 60 seconds without asking me again."
<code>no-cache</code>	"You can store this, but you MUST validate via ETag before you use it."
<code>no-store</code>	"Extremely sensitive data. NEVER save this to disk, not even for a microsecond."

4. The [ResponseCache] Attribute Deep Dive

Applying this attribute to a controller directly manipulates the headers sent to the client.

- **Duration:** (Integer) The lifespan of the cache in seconds. *Generates:* Cache-Control: max-age={Duration}
- **Location:** (Enum) Dictates WHO is allowed to store the cache.
 - Any: Generates public. Browsers and CDNs can store it.
 - Client: Generates private. ONLY the user's specific browser can store it.
 - None: Generates no-cache. Must revalidate with the server.
- **NoStore:** (Boolean) The nuclear option for sensitive data. *Generates:* Cache-Control: no-store. Overrides Duration and Location completely.
- **VaryByHeader:** (String) Caches different versions of the response based on a client header (e.g., "Accept-Language").
- **VaryByQueryKeys:** (String Array) Caches different versions based on URL parameters (e.g., ?page=1 vs ?page=2). *Note: Requires app.UseResponseCaching() middleware to function.*

5. Enterprise Architecture: Global Cache Profiles

Writing cache attributes on every single controller violates the DRY (Don't Repeat Yourself) principle. Senior developers define Cache Profiles centrally in Program.cs.

```
// Program.cs
builder.Services.AddControllers(options =>
{
    // Profile 1: For Public Static Data (Like a catalog of products)
    options.CacheProfiles.Add("PublicData", new CacheProfile()
    {
        Duration = 3600, // 1 hour
        Location = ResponseCacheLocation.Any
    });

    // Profile 2: For Private User Data (Like a user's shopping cart)
    options.CacheProfiles.Add("PrivateUserData", new CacheProfile()
    {
        Duration = 60, // 1 minute
        Location = ResponseCacheLocation.Client // PRIVATE!
    });
});
```

Once defined, apply the profile cleanly to your endpoints:

```
[ApiController]
[Route("api/[controller]")]
public class CatalogController : ControllerBase
{
    // Inherits the 1-hour, Public caching rule
    [HttpGet]
    [ResponseCache(CacheProfileName = "PublicData")]
    public IActionResult GetCatalog()
    {
        return Ok(...);
    }
}
```